

Silent event loss in Prophesee's video-to-event sample script

A truncation bug in OpenEB 5.2.0 sample code, how it was found, and the replacement converter written to avoid it.

Summary

- Prophesee ships `viz_video_to_event_simulator.py` as a **display** tool that also accepts an `-o` flag to save events to a DAT file.
- Its display loop waits until 50,000 events have accumulated before drawing, then keeps exactly that many. File output rides along inside that loop, so it inherits both behaviours.
- Result: any conversion that produces events sparsely loses most of the clip, with no error and no warning.
- Fixed by writing a separate converter that drives the same `EventSimulator` and writes every event, with no thresholding or buffering.

How it was found

- **Symptom.** A verification pass reported that a converted clip's events ended well before the video did, and the final frames contained no events at all.
- **First measurement.** The output file was 400,109 bytes. Subtracting the ~109-byte DAT header and dividing by 8 bytes per event gave exactly 50,000 — a suspiciously round number matching the default of `--n_events`.
- **Ruled out a one-off.** Re-running the same conversion produced 400,107 bytes — again 50,000 events. Systematic, not an interrupted run.
- **Located the parameter.** `findstr /n "n_events" <script>` showed three uses: the argument definition, a `FixedCountBuffer` construction, and a comparison inside the main loop.
- **Read the loop.** Printing the surrounding lines revealed the two mechanisms below.
- **Confirmed against the full file.** Reading the whole script showed `writer.write(events)` sits *after* the skip, inside the display block — so nothing below the threshold can ever reach the file.

The mechanism

The relevant part of Prophesee's loop:

```
for img, ts in image_stream:
    total = simu.image_callback(img, ts)

    if total < n_events: # (1) skip until 50,000 accumulate
        continue

    events = simu.get_events()
    simu.flush_events()
    events = fixed_buffer(events) # (2) keep only 50,000
    ...
    if args.output:
        writer.write(events) # reached only past (1)
```

- **(1) The skip.** Events accumulate in the simulator until the count crosses 50,000. Everything before that point is flushed without being written.
- **(2) The buffer.** Even when the threshold is crossed, `FixedCountBuffer` keeps exactly 50,000 events and discards any excess.
- **No final drain.** The loop ends without pulling whatever the simulator still holds, so the tail of every clip is dropped regardless of density.

Severity scales inversely with event rate. A dense stream crosses the threshold repeatedly and loses only its final partial buffer (under 1% in testing). A sparse stream may cross it once in the entire clip, losing roughly 30%, concentrated at the end.

This makes the bug especially damaging when comparing simulator presets, because the presets differ mainly in event count — the exact variable the bug is sensitive to. Quiet presets are penalised hardest, so the comparison is not like for like.

Why the code is written this way

- The script's docstring describes it as displaying events and buffering a FixedTime/CountBufferizer. It is a viewer, and is documented as one.
- Fixed-count accumulation is the right choice *for display*: it keeps image brightness stable whether the scene is busy or quiet. Fixed-time windows would flicker between empty and saturated.
- The `-o` flag appears to have been added later and wired into the display path rather than given its own. Its help text says only that it writes events to a DAT file, with no mention of the buffering.
- Separately, `--leak_rate_hz` is accepted, documented and shown in `--help`, but never passed to the `EventSimulator` constructor. It silently does nothing.

The replacement converter

Prophesee's script was left untouched; a separate `convert_video_to_events.py` was written alongside it. The heavy lifting is in the library (`metavision_core_ml.video_to_event.simulator`), not the sample, so the replacement calls the same class with the same parameters and produces the same DAT format.

What differs:

- **No threshold.** Every frame's events are drained and written immediately.
- **No buffer.** `FixedCountBuffer` is not used at all. Slicing into windows is the reader's job — a DAT file is a timestamped stream and should not have a windowing choice baked in at write time.
- **Final drain.** After the last frame, the simulator is emptied once more so the tail is not lost.
- **No display code.** No OpenCV windows, so no `--no_display` flag.
- `--leak_rate_hz` **is actually passed** to the simulator.
- **Reports what it did.** Prints frames processed, events written, last timestamp and event rate — so a truncated or empty conversion is visible immediately rather than silent.

Core of the loop:

```
for img, ts in stream:
    simulator.image_callback(img, ts)

    events = simulator.get_events()
    simulator.flush_events()

    if len(events):
        writer.write(events)

# drain whatever remains after the last frame
events = simulator.get_events()
simulator.flush_events()
if len(events):
    writer.write(events)
```

Verification of the fix

- Same clip, same parameters, both scripts. The replacement produced more events and a last-event timestamp matching the video duration, where the original stopped short.
- Event count was cross-checked against file size: $(\text{bytes} - \text{header}) \div 8$ matched the reported count in both cases.
- A label-alignment pass over the corrected file confirmed the recovered events were valid data, not padding — the previously empty final frames scored among the strongest in the clip.

Takeaways

- Sample scripts are demonstrations of a library, not production tools. The library was correct throughout; only the wrapper around it discarded data.
- A round number in an output size is worth investigating. 50,000 events was the first and strongest clue.
- Silent data loss is the dangerous kind. The conversion exited cleanly with status 0 every time.
- Any pipeline built on `viz_video_to_event_simulator.py -o` is affected, in proportion to how sparse its output is.